

MIRA Flutter Project Guide

Folder structure, MVVM data flow, routes, Firebase/Firestore collections, Cloudinary image flow, aur current app behavior.

Item	Current value
App display name	MIRA
Root Flutter class	AveroApp
Android package ID	com.example.flutter_ecommerce_app
Firebase project	avero-147cb
UI foundation	Flutter Material + GetMaterialApp
Architecture	MVVM. GetX Controller acts as ViewModel.
Report date	October 2, 2026

Note: Ye PDF current project state ke hisaab se hai. Is mein code modify nahi kiya gaya; sirf explanation/documentation generate hui hai.

1. High Level Architecture

Project ka main pattern MVVM hai. View UI render karti hai, Controller/ViewModel screen state aur user actions handle karta hai, Repository business-friendly API deti hai, Service Firebase/Cloudinary jaisi external layer se baat karti hai, aur Model data ka shape define karta hai.

```
View -> Controller/ViewModel -> Repository -> Service -> Firebase/Cloudinary
```

- View files sirf UI aur user interaction rakhti hain.
- Controller files GetX reactive state, loading, validation aur navigation handle karti hain.
- Repository files feature-friendly methods deti hain, jaise addVendorProduct ya createOrder.
- Service files low-level FirebaseAuth, Firestore, Cloudinary upload aur storage calls karti hain.
- Model files Firestore documents ko Dart objects mein convert karti hain.

2. lib Folder Inventory

Folder/File	Kaam
lib/main.dart	App startup point. Firebase initialize hota hai, phir runApp(AveroApp()) call hoti hai.
lib/firebase_options.dart	FlutterFire generated Firebase client config. Is mein public Firebase app options hote hain, private server secret nahi hota.
lib/core	App-wide shared layer: app shell setup, routes, theme, validators, responsive helpers aur reusable widgets.
lib/data	Data layer: models, repositories aur services. Ye UI ko direct Firebase details se isolate karta hai.
lib/modules	Feature-first screens. Har feature apni binding, controller, view aur widgets ke sath organized hai.

3. core Folder

Path	Responsibility
core/app.dart	AveroApp class. GetMaterialApp configure karta hai: title MIRA, theme, routes, initial route aur UnfocusOnTap wrapper.
core/bindings	InitialBinding app start par global dependencies register karti hai, jaise services aur repositories.
core/constants	Reusable constants: spacing/layout, Cloudinary config, aur Cloudinary-hosted asset URL mapping.
core/routes	Centralized route names aur GetPage mapping. App mein navigation hardcoded strings se bachti hai.
core/theme	Colors, palette, text styles, ThemeData, aur theme mode controller. UI Material base par hai, generic default Material look nahi.
core/utils	Validators, responsive helpers, aur product image URL resolver.
core/widgets	Reusable widgets: snackbar, loading/empty/error states, auth text field, logo, product image, responsive field pair, unfocus wrapper.

4. data Folder

Data folder project ki Firebase/Cloudinary access layer hai.

Folder	Files / Purpose
data/models	address_model, cart_item_model, category_model, order_models, product_model, user_model, vendor_model, wishlist_item_model. Ye Firestore document ko Dart model banate hain.

Folder	Files / Purpose
data/repositories	address_repository, auth_repository, cart_repository, order_repository, product_repository, user_repository, vendor_repository, wishlist_repository. Controllers inhi methods ko call karte hain.
data/services	auth_service FirebaseAuth/Google Sign-In ke liye; firestore_service Firestore reads/writes/transactions ke liye; cloudinary_service image upload ke liye; theme_storage_service theme preference ke liye.

Important services:

- AuthService: email/password login, signup, Google sign-in, password reset, sign out, recent-login check, account deletion.
- FirestoreService: users, categories, products, cart, wishlist, addresses, orders aur vendors collections access karta hai.
- CloudinaryService: product/profile images ko Cloudinary unsigned upload preset avero_products ke through upload karta hai.

5. modules Folder

Module	Kaam
splash	Startup screen. Auth/profile state check karke Login ya Home route decide karti hai.
auth/login	Email/password aur Google sign-in UI + controller.
auth/signup	Customer account create karta hai aur users/{uid} profile banata hai.
auth/forgot_password	Password reset email flow.
app_shell	Main customer shell: Home, Collections, Cart, Profile tabs.
home	Home catalog. Active products ko category sections mein group karke show karta hai.
collection	Category/product listing screen.
product_detail	Selected product ki detail, vendor name, wishlist aur add-to-cart actions.
wishlist	Current user wishlist read/remove.
cart	Current user cart items, quantity changes aur checkout navigation.
checkout	Delivery address, cart summary, total aur place order. Payment method UI-only hai.
profile	Profile view/edit, vendor entry, sign out, delete account.
vendor/become_vendor	Customer ko vendor profile create karne ka form.
vendor/dashboard	Vendor home/dashboard area.
vendor/products	Vendor products list and product create/edit form.
vendor/orders	Vendor ko un orders ka list jahan uski products included hain.

6. Route Map

Route	Screen	Binding
/	SplashView	SplashBinding
/login	LoginView	LoginBinding
/signup	SignupView	SignupBinding
/forgot-password	ForgotPasswordView	ForgotPasswordBinding
/home	AppShellView	AppShellBinding
/product-detail	ProductDetailView	ProductDetailBinding

Route	Screen	Binding
/wishlist	WishlistView	WishlistBinding
/cart	CartView	CartBinding
/checkout	CheckoutView	CheckoutBinding
/collection	CollectionView	CollectionBinding
/become-vendor	BecomeVendorView	BecomeVendorBinding
/vendor-dashboard	VendorDashboardView	VendorDashboardBinding
/vendor-products	VendorProductsView	VendorProductsBinding
/vendor-orders	VendorOrdersView	VendorOrdersBinding
/vendor-product-form	VendorProductFormView	VendorProductFormBinding

7. Core Data Flows

Startup

main.dart -> WidgetsFlutterBinding.ensureInitialized -> Firebase.initializeApp(DefaultFirebaseOptions.currentPlatform)
-> runApp(AveroApp) -> GetMaterialApp -> SplashController -> Auth/Profile check -> Login or Home.

Login

LoginView -> LoginController -> AuthRepository -> AuthService -> FirebaseAuth/GoogleSignIn ->
UserRepository.ensureCustomerProfileExists -> users/{uid} -> AppShell.

Signup

SignupView -> SignupController -> AuthRepository -> AuthService.createUserWithEmailPassword ->
UserRepository.createInitialCustomerProfile -> users/{uid}.

Home catalog

HomeView -> HomeController -> ProductRepository -> FirestoreService -> categories/products. Products active +
valid vendorId ke sath category sections mein show hoti hain.

Product detail and cart

ProductCard/ProductDetailView -> CartController -> CartRepository -> FirestoreService -> users/{uid}/cart/{productId}.
Cart stored quantity rakhta hai; product detail products collection se aati hai.

Wishlist

WishlistView -> WishlistController -> WishlistRepository -> FirestoreService -> users/{uid}/wishlist/{productId}.

Checkout/order

CheckoutView -> CheckoutController -> AddressRepository + CartController + OrderRepository -> Firestore
transaction. Cart docs and product docs read hote hain, order create hota hai, stock/soldCount update hota hai, cart
clear hota hai.

Become vendor

BecomeVendorView -> BecomeVendorController -> VendorRepository -> Firestore transaction. vendors/{uid} create
hota hai aur users/{uid}.role vendor hota hai.

Vendor product create/edit

VendorProductFormView -> VendorProductFormController -> CloudinaryService upload -> ProductRepository ->
FirestoreService -> products/{productId}. Product vendorId aur vendorName save karta hai.

Delete account

ProfileView -> ProfileController -> AuthRepository.deleteCurrentAccount -> recent-login check -> UserRepository.deleteUserOwnedData -> delete vendor products/cart/wishlist/addresses/vendor/user docs -> FirebaseAuth user delete.

8. Firestore Collections

Collection	Structure / fields	Purpose
users/{uid}	uid, email, displayName, photoUrl, role, vendorId, isActive, createdAt, updatedAt	Customer/vendor profile. FirebaseAuth user ke sath linked document.
users/{uid}/wishlist/{productId}	productId, createdAt	User-specific wishlist. Product details products collection se loaded hoti hain.
users/{uid}/cart/{productId}	productId, quantity, addedAt, updatedAt	User-specific cart. Quantity yahan store hoti hai.
users/{uid}/addresses/{addressId}	fullName, phone, address lines, city, state, postalCode, country, isDefault, timestamps	Checkout delivery addresses.
categories/{categoryId}	name, imageUrl, sortOrder, isActive	Catalog categories. App currently read-only use karta hai.
products/{productId}	name, description, price, imageUrl, categoryId, categoryName, vendorId, vendorName, stock, compareAtPrice, isActive, soldCount, timestamps	Vendor-created products. Home/collection/product detail isi se read karte hain.
vendors/{uid}	vendorId, ownerId, storeName, ownerName, email, phone, description, isActive, timestamps	Vendor store profile. Document id owner uid ke equal hai.
orders/{orderId}	orderId, customerId, vendorIds, items snapshots, itemCount, deliveryAddress, subtotal, total, status, timestamps	Placed order. Payment abhi implemented nahi, status pending hai.

9. Firestore Rules Summary

- Default rule deny hai: jo explicitly allow nahi, wo blocked hai.
- users/{uid}: user sirf apna profile read/create/update/delete kar sakta hai.
- User safe profile update mein sirf displayName, photoUrl, updatedAt change ho sakte hain.
- Role/vendorId conversion sirf become-vendor transaction ke valid getAfter checks ke sath hoti hai.
- wishlist, cart, addresses subcollections sirf owner uid ke liye open hain.
- categories read signed-in users ke liye only active categories par allowed hai; writes disabled hain.
- products read active products ya apni vendor products ke liye allowed hai. Create/update/delete vendor owner ke liye restricted hai.
- Order placement transaction stock reduce aur soldCount increase kar sakti hai. Products ka vendorId client update nahi kar sakta.
- orders read customer ya included vendor kar sakta hai. Order update/delete disabled hai.
- vendors read/delete own vendor doc ke liye hai; create self vendor conversion transaction ke through validate hota hai; update disabled hai.

10. Image and Cloudinary Flow

Project ab product/profile images ke liye local permanent image storage use nahi karta. Temporary picked file sirf preview ke liye hoti hai. Save par image Cloudinary par upload hoti hai, phir returned secure URL Firestore document mein imageUrl/photoUrl ke andar save hota hai.

Area	Flow
Product images	Vendor selects image -> local preview -> Cloudinary upload folder avero/products -> URL saved in products/{productId}.imageUrl.
Profile image	User selects image -> Cloudinary upload folder avero/profiles -> URL saved in users/{uid}.photoUrl.

Area	Flow
Home/product UI	ProductImage widget productImageUrl(product) se URL resolve karke network image render karta hai.
Config	Cloud name dtwghsqhnp, upload preset avero_products. No private Cloudinary API secret in Flutter app.

11. Product Visibility Rules in App

- Customer side par product tabhi visible hai jab isActive true ho.
- Product name empty nahi hona chahiye.
- vendorId empty nahi hona chahiye; warna product customer catalog mein hidden rahegi.
- New vendor product create par current vendor storeName product.vendorName ke andar denormalize hota hai.
- Home page product ko categoryName/categoryId ke hisaab se sections mein group karta hai.
- Product card order: title, vendor name, price.
- Product detail screen par bhi vendor name show hota hai.

12. Important Notes / Current Limits

- Payment Method screen UI-only hai. Stripe/PayPal/COD/payment verification abhi implemented nahi.
- Order success language paid success claim nahi karti; order status pending hai.
- FirebaseAuth dashboard se user manually delete karne par Firestore cascade automatically nahi chalega. App ke Delete Account flow se delete karna hoga. Automatic backend cascade ke liye future Cloud Function chahiye.
- Firestore users/products cleanup previously manual command se kiya gaya tha; normal app flow ab user-owned delete path use karta hai.
- Product/vendor image URLs Cloudinary par depend karte hain. Agar preset/cloud config change ho to upload fail ho sakta hai.
- Admin approval, vendor moderation, product review workflow aur payment backend future phases hain.

13. Quick Mental Model

Aap project ko teen layers mein soch sakte hain: modules UI aur screen behavior, data repositories/services actual data movement, aur core shared app foundation. Firestore ke documents app state ka source hain; Cloudinary sirf images ka public URL source hai; FirebaseAuth login identity deta hai.

Example: Vendor product add karta hai. UI form Controller ko data deta hai. Controller image upload ke liye CloudinaryService call karta hai. Phir ProductRepository product model ko FirestoreService ke through products collection mein save karta hai. HomeController active products read karta hai aur customer app mein category section ke neech show karta hai.